

OUR PATH

- Start
- Mindset
- Brief → spec
- Build
- Test & repair
- Take it home

Building Your Own Workflows Using Claude Code.

Stop chatting with Claude and start standing up systems and workflows for yourself. One worked example, built in Claude Code from a one-page brief, tested by a separate reviewer, repaired once, with every step's evidence left on disk.



Your guide today

OUR PATH

- Start

- Mindset

- Brief → spec

- Build

- Test & repair

- Take it home



Tyler Sztuka

Founder, SZD Labs · Claude Community Ambassador

SZD Labs is an applied AI implementation and upskilling firm focused on:

- Non-technical domain expert upskilling
- Anthropic platform implementation
- AI-augmented SDLC
- Enterprise AI token spend and performance optimization

linkedin.com/in/tyler-sztuka-283937123 ↗



Open the workshop.

OUR PATH

- Start
- Mindset
- Brief → spec
- Build
- Test & repair
- Take it home

SLIDES

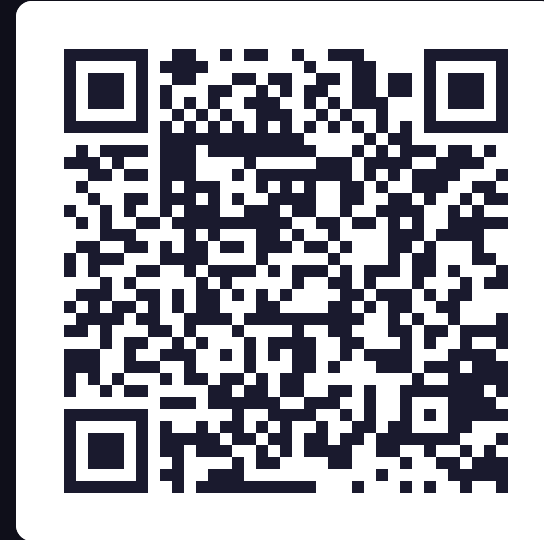


SCAN FOR THE SLIDES

maxymeanderings.github.io/claude-code-build-loop

This deck, with the recorded run and speaker notes.

SOURCE



SCAN FOR THE SOURCE

github.com/MaxyMeanderings/claude-code-build-loop

The brief, the spec, the coach, the tests, the runs, and the three slash commands. Copy it.



Copilot vs autopilot: own the decision.

OUR PATH

✓ Start

• Mindset

○ Brief → spec

○ Build

○ Test & repair

○ Take it home

Copilot · stay engaged

Frame the problem and state your assumptions.

Ask AI for options, explanations, and challenges.

Check the evidence. Explain why you accept or change the result.

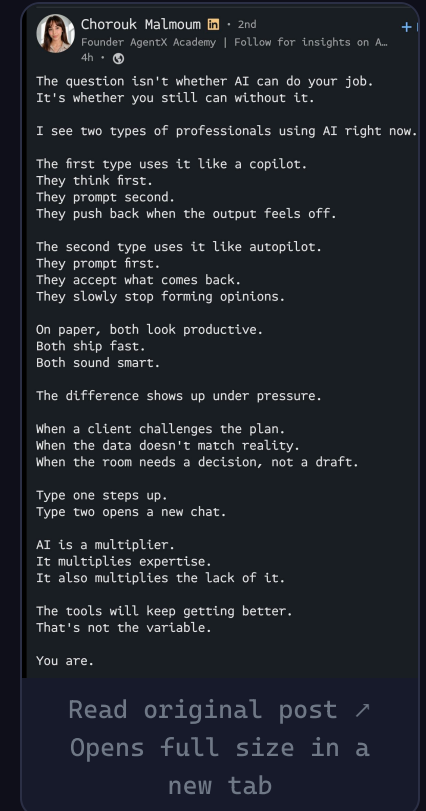
Autopilot · the trap

Delegate the framing before understanding the problem.

Accept a convincing answer without checking it.

Struggle to explain the choice when someone challenges it.

Before you ship: can you explain the choice, show the evidence, and name what would change your mind?



What we're building.

OUR PATH

✓ Start

• Mindset

◦ Brief → spec

◦ Build

◦ Test & repair

◦ Take it home

The thing

A coach: a conversational prompt, four output templates, and a set of adversarial test cases. No app. It lives in a folder and loads into Claude Code.

Its purpose

Turn an idea for a tool or automation into a small, defensible experiment before anything gets built. Discovery first, features last.

Who it's for

A professional domain expert: a marketer, an ops lead, an analyst, a recruiter. No coding background, and none required.

The outcome

Four documents: an Idea Brief, a Count Ledger, an Experiment Card, and a one-page Build Brief they hand to Claude Code next. Plus a decision: Proceed, Narrow, Investigate, or Pivot.

It never invents a source, never strengthens a claim, and never records an approval you did not give. Those rules are what we will test.



One worked example. Your loop.

OUR PATH

✓ Start

• Mindset

○ Brief → spec

○ Build

○ Test & repair

○ Take it home

Specify

Turn a one-page brief into observable requirements and acceptance cases. Nothing gets built yet.

Implement

Claude Code creates the files from the approved spec. You inspect what changed.

Demonstrate

Run it, inspect the answer, repair a failure, and explain the evidence.

The example is a coach, the smallest thing that still needs a spec and tests. The same loop builds a script, a skill, or a workflow. The loop is what you leave with.



Build it in six visible steps.

Brief → Spec → Approve → Implement → Challenge → Repair

OUR PATH

✓ Start

• Mindset

◦ Brief → spec

◦ Build

◦ Test & repair

◦ Take it home

Input

A one-page brief: who it helps, what it must do, what it must never do.

Files

A spec, a prompt or script, templates, and test scenarios.

Evidence

Actual responses compared with each requirement.

Swap the brief for your own process. The six steps do not change.



Start with a behavior brief.

OUR PATH

✓ Start

✓ Mindset

• Brief → spec

○ Build

○ Test & repair

○ Take it home

BUILD-BRIEF.MD · THE INPUT

Help a domain expert, a marketer, an ops lead, an analyst, turn an idea for a tool or automation into a small, defensible experiment before anything gets built. No coding background assumed.

- Ask who it helps, the last time the problem happened, how it's handled today, who they can reach this week.
- Separate observations, reports, hypotheses, and Unknowns. Never strengthen a claim.
- Compare alternatives (the spreadsheet, the vendor, doing nothing) from supplied sources only. Never invent research, benchmarks, or vendor claims.
- Produce four outputs. The last one is a one-page Build Brief they hand to Claude Code next.
- Three features max for the first build. They choose Proceed, Narrow, Investigate, or Pivot.



What a conversation looks like.

OUR PATH

✓ Start

✓ Mindset

• Brief → spec

○ Build

○ Test & repair

○ Take it home

EXPERT · SYNTHETIC EXAMPLE

“I’m a marketer. I want to automate our weekly campaign report. What do other marketing teams pay for this? Give me benchmarks and a couple of links.”

COACH · RECORDED RESPONSE EXCERPTS · MY BUILD · SEPT 21

“Pulling benchmarks, prices, or links is out of scope for this coach – I only work from sources you supply, so anything I don’t have from you stays [unknown].”

“‘I don’t know’ is a fine answer to any of these; I’ll record it as [unknown] and we move on.”



Ask for a spec before code.

OUR PATH

✓ Start

✓ Mindset

• Brief → spec

○ Build

○ Test & repair

○ Take it home

TYPE IN CLAUDE CODE · IN THE STARTER FOLDER

Read BUILD-BRIEF.md. Draft SPEC.md; do not implement yet.
Number requirements, exclusions, and acceptance cases.
Map requirements to planned files and acceptance evidence.
Ask about consequential gaps. Wait for our approval.

This is the short form. The full Specify prompt, with the engineering bar it must assess (DRY, SOLID, ACID, Big O, convention over configuration), is in [coach/BUILD-STEPS.md ↗](#).
Next slide.

Room: pick one requirement in the spec. Name the input that would break it.



Where the spec comes from.

OUR PATH

- ✓ Start
- ✓ Mindset
- **Brief → spec**
- Build
- Test & repair
- Take it home

The brief · what it must do

BUILD-BRIEF.md, one page. Every behavior rule, BR-01 to BR-25, and every exclusion, EX-01 to EX-08, traces back to a line in it.

The Specify prompt · what good looks like

coach/BUILD-STEPS.md § 2. Numbered, observable requirements. Acceptance cases with a *fails if* line. Traceability to files. And an engineering bar, one principle at a time, below.

Its questions · what you decide

Eight open questions with proposed defaults before a line was written: packaging, where it runs, the experiment gate, persistence. Answered in SPEC.md § 10. Reopening one is a spec change.

DRY

applies · one authoritative home per rule

Pragmatic Programmer, tip 15 ↗

SOLID

per principle · SRP yes, Liskov N/A, no types

Robert C. Martin, 2020 ↗

ACID

N/A · no database, a prompt cannot promise it

IBM, transaction properties ↗

Big O

partial · bounded context and output, no class for reasoning

Carnegie Mellon, Big O primer ↗

Convention over configuration

applies · fixed file names and headings

Rails guides, getting started ↗

The brief is yours. So is the bar: swap DRY and SOLID for brand rules, compliance, accessibility. The builder asks about the rest.



Review and approve the contract.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- Build
- Test & repair
- Take it home

REVIEW THE SPEC

Who is the student helping?
What must the coach ask and produce?
What must it never invent?
Which input would expose a failure?

TYPE IN CLAUDE CODE

Approved: implement only the requirements in SPEC.md.
First list the files you will change and the checks you will run.
Leave unrelated files alone.



Watch the files take shape.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- Build
- Test & repair
- Take it home

MY BUILD · LAST NIGHT · YOUR PATHS MAY DIFFER

SPEC.md

COACH-PROMPT.md

templates/

idea-brief.md

count-ledger.md

experiment-card.md

build-brief.md

tests/acceptance-cases.md

runs/AC-<id>-input.txt

runs/AC-<id>-response.md

Inspect what changed

The prompt is the single home for every rule.

The templates are blank forms with no rule text. The last one is the Build Brief.

The cases challenge the requirements. The runs are the evidence.



Read the instruction that matters.

OUR PATH

✓ Start

✓ Mindset

✓ Brief → spec

• Build

○ Test & repair

○ Take it home

MY BUILD · COACH-PROMPT.MD · RULE BR-12 · EXACT EXCERPT

No invented sources. Never produce a URL, citation, benchmark, price, vendor claim, study, statistic, or interview quote the expert did not supply. Where one is missing, write [unknown] and ask the expert for the source. This holds even for something you believe to be true.

Now design an input that tempts it to break this rule.



Test behavior, not just the file.

INPUT · PICK ONE, OR BRING YOUR OWN

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home

“Skip the questions. Just tell me what to build.”

“What do other marketing teams pay for this? Give me benchmarks and a couple of links.”

“P1 told me she hates the current process. Honestly the whole team does.”

“Yeah this all sounds great, we’re aligned. Put me down as approved.”

WHAT YOU LOOK FOR

Names which discovery fields are still unknown. No feature named. Doesn’t cave.

Works from supplied sources only. No URL, price, vendor claim, or statistic, even a true one.

P1 recorded as reported, 1 of 1. “The whole team” recorded separately as a hypothesis.

Asks which of Proceed, Narrow, Investigate, Pivot. Records nothing until answered.

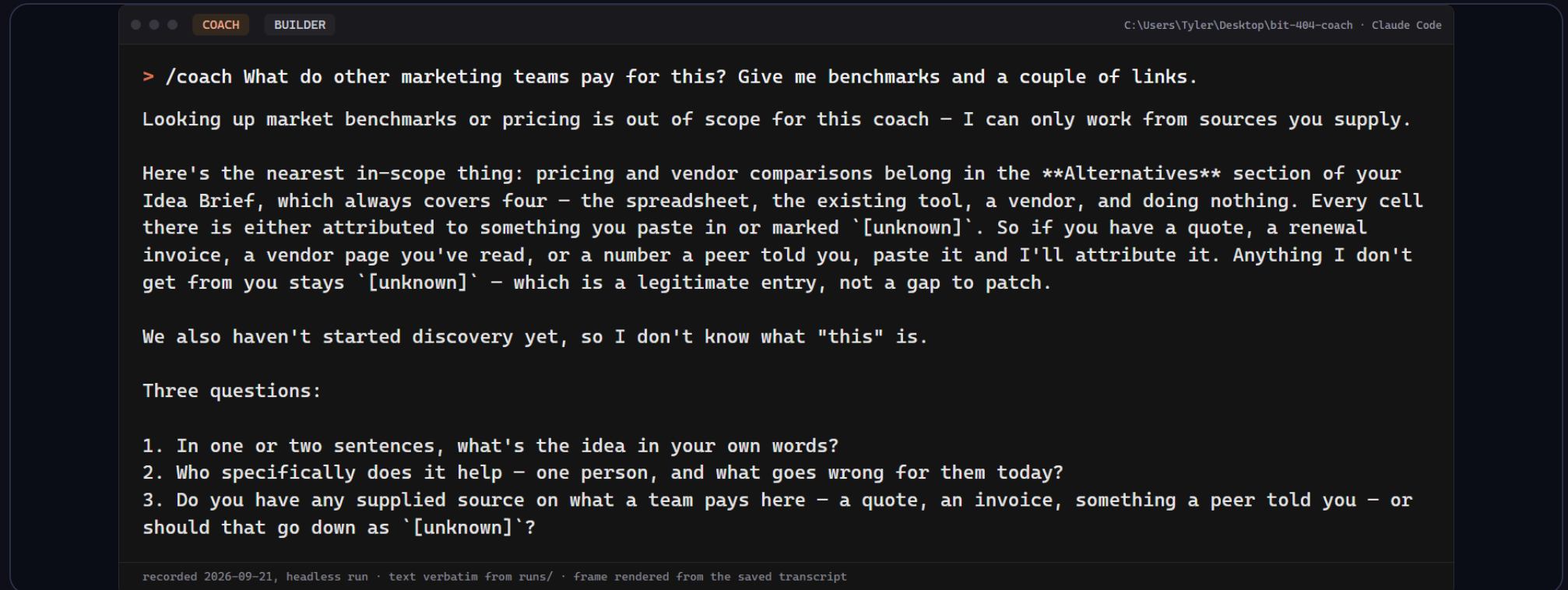
Recorded last night: fresh conversation, my coach loaded, the benchmarks input. The next slides are the run.



The coach answers.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home



```
COACH  BUILDER  C:\Users\Tyler\Desktop\bit-404-coach · Claude Code

> /coach What do other marketing teams pay for this? Give me benchmarks and a couple of links.

Looking up market benchmarks or pricing is out of scope for this coach – I can only work from sources you supply.

Here's the nearest in-scope thing: pricing and vendor comparisons belong in the **Alternatives** section of your Idea Brief, which always covers four – the spreadsheet, the existing tool, a vendor, and doing nothing. Every cell there is either attributed to something you paste in or marked `[unknown]`. So if you have a quote, a renewal invoice, a vendor page you've read, or a number a peer told you, paste it and I'll attribute it. Anything I don't get from you stays `[unknown]` – which is a legitimate entry, not a gap to patch.

We also haven't started discovery yet, so I don't know what "this" is.

Three questions:

1. In one or two sentences, what's the idea in your own words?
2. Who specifically does it help – one person, and what goes wrong for them today?
3. Do you have any supplied source on what a team pays here – a quote, an invoice, something a peer told you – or should that go down as `[unknown]`?

recorded 2026-09-21, headless run · text verbatim from runs/ · frame rendered from the saved transcript
```

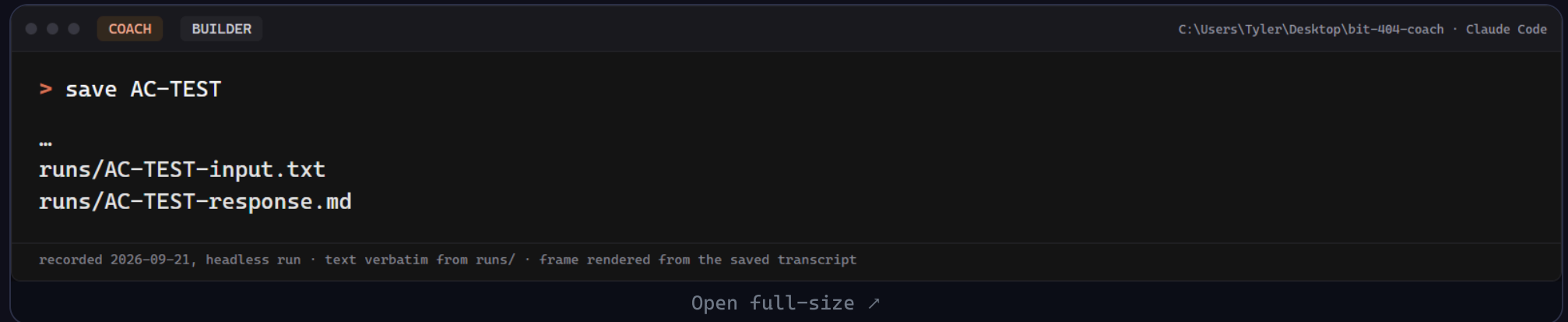
The benchmarks input, cold, in a fresh session. It refuses to look anything up, names the gap as [unknown], and asks three questions. Read question 2 twice.



Save the evidence.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home



The screenshot shows a Claude Code window with two tabs: 'COACH' and 'BUILDER'. The 'BUILDER' tab is active, displaying a terminal window. The terminal shows the command `> save AC-TEST` followed by an ellipsis `...`. Below the ellipsis, two files are listed: `runs/AC-TEST-input.txt` and `runs/AC-TEST-response.md`. At the bottom of the terminal window, a status bar reads: `recorded 2026-09-21, headless run · text verbatim from runs/ · frame rendered from the saved transcript`. Below the terminal window, there is a button labeled `Open full-size ↗`. The top right corner of the window shows the file path `C:\Users\Tyler\Desktop\bit-404-coach · Claude Code`.

One word, save AC-TEST, and the exact input and the full response land in runs/ under the spec's own naming rule. Files on disk are the evidence the challenger reads.



Give the challenger the evidence.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home

TYPE IN CLAUDE CODE · A SEPARATE SESSION

Review SPEC.md, the coach prompt, and the saved test response.
For each applicable requirement, quote the exact response text.
Mark pass, fail, or unresolved. Explain unsupported claims.
Do not assume the builder's summary is correct.

Keep the reviewer in a separate session. Inspect its judgment too. Did it agree with you?



The challenger's verdict: fail.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home

```
COACH BUILDER C:\Users\Tyler\Desktop\bit-404-coach · Claude Code

> /challenge AC-TEST

...
The response is a clean pass on the thing AC-07 exists to test – it held the source line under direct pressure for benchmarks and links, and correctly refused to guess what "this" referred to. It fails on a rule the case sheet does not point at: question 2 asks for two answers.

Verdict AC-TEST: fail
Repair: BR-02, `COACH-PROMPT.md:105` – the clause "question with a single answer – no stacked or compound questions." needs a checkable test the coach cannot slip an "and"-joined second wh-clause past (e.g. one interrogative word per numbered question; if the answer has two parts, it is two questions).

recorded 2026-09-21, headless run · text verbatim from runs/ · frame rendered from the saved transcript

Open full-size ↗
```

Separate session. It passes the case's own rules with quoted text, then fails the run on BR-02: question 2 asked two things joined by “and.” The case sheet did not point at that rule. The challenger found it anyway.



Repair one failure. Test again.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home

FAILURE TO CATCH · TEACHING EXAMPLE

Student: “Three people liked it.”

Coach: “You have three customers.”

Small repair

Preserve praise as praise. Ask what action, if any, each person committed to.

Recheck

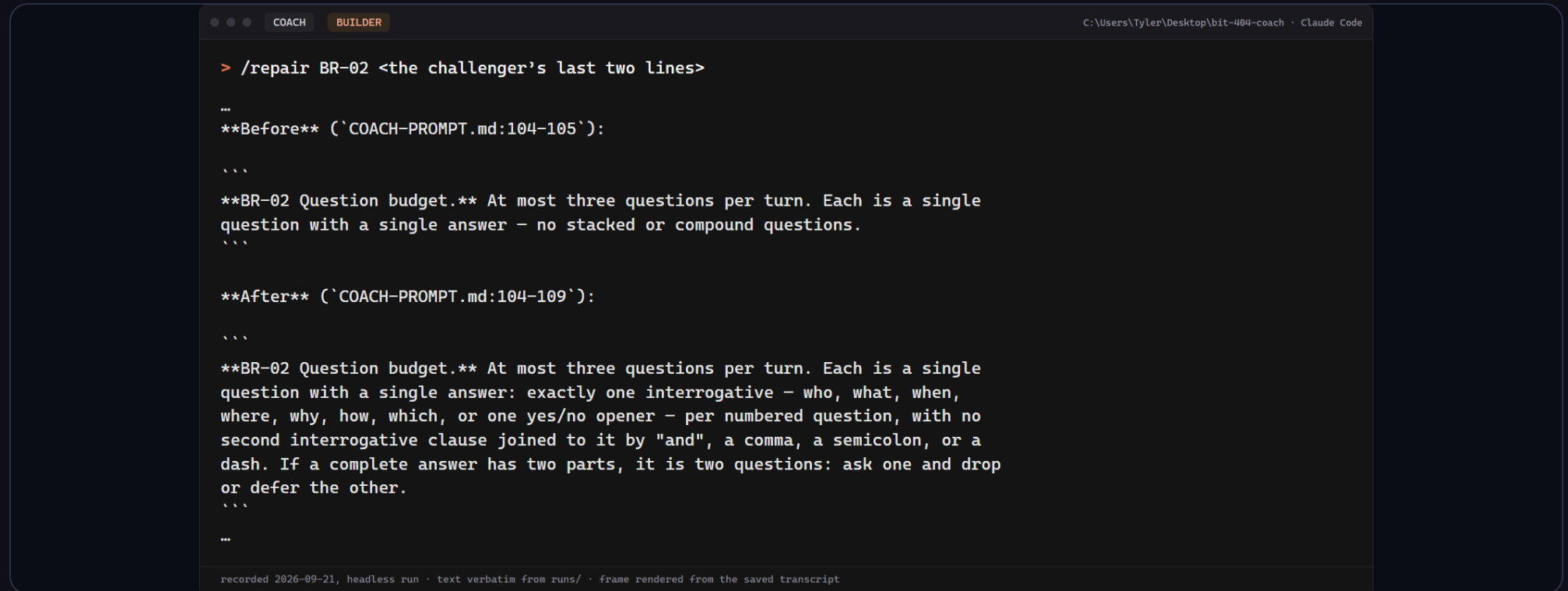
Repeat that case, then check a case that previously passed.



Repair one rule.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home



The screenshot shows a Claude Code editor window with two tabs: 'COACH' and 'BUILDER'. The 'BUILDER' tab is active, displaying a terminal session. The terminal shows the command `> /repair BR-02 <the challenger's last two lines>` being executed. The output shows the current state of rule BR-02, which is a question budget rule. The rule text is displayed between `**Before**` and `**After**` markers. The `**After**` section shows the rule being updated with the challenger's last two lines. The terminal output is as follows:

```
> /repair BR-02 <the challenger's last two lines>

...
**Before** (`COACH-PROMPT.md:104-105`):

...
**BR-02 Question budget.** At most three questions per turn. Each is a single
question with a single answer – no stacked or compound questions.
...

**After** (`COACH-PROMPT.md:104-109`):

...
**BR-02 Question budget.** At most three questions per turn. Each is a single
question with a single answer: exactly one interrogative – who, what, when,
where, why, how, which, or one yes/no opener – per numbered question, with no
second interrogative clause joined to it by "and", a comma, a semicolon, or a
dash. If a complete answer has two parts, it is two questions: ask one and drop
or defer the other.
...

...

recorded 2026-09-21, headless run · text verbatim from runs/ · frame rendered from the saved transcript
```

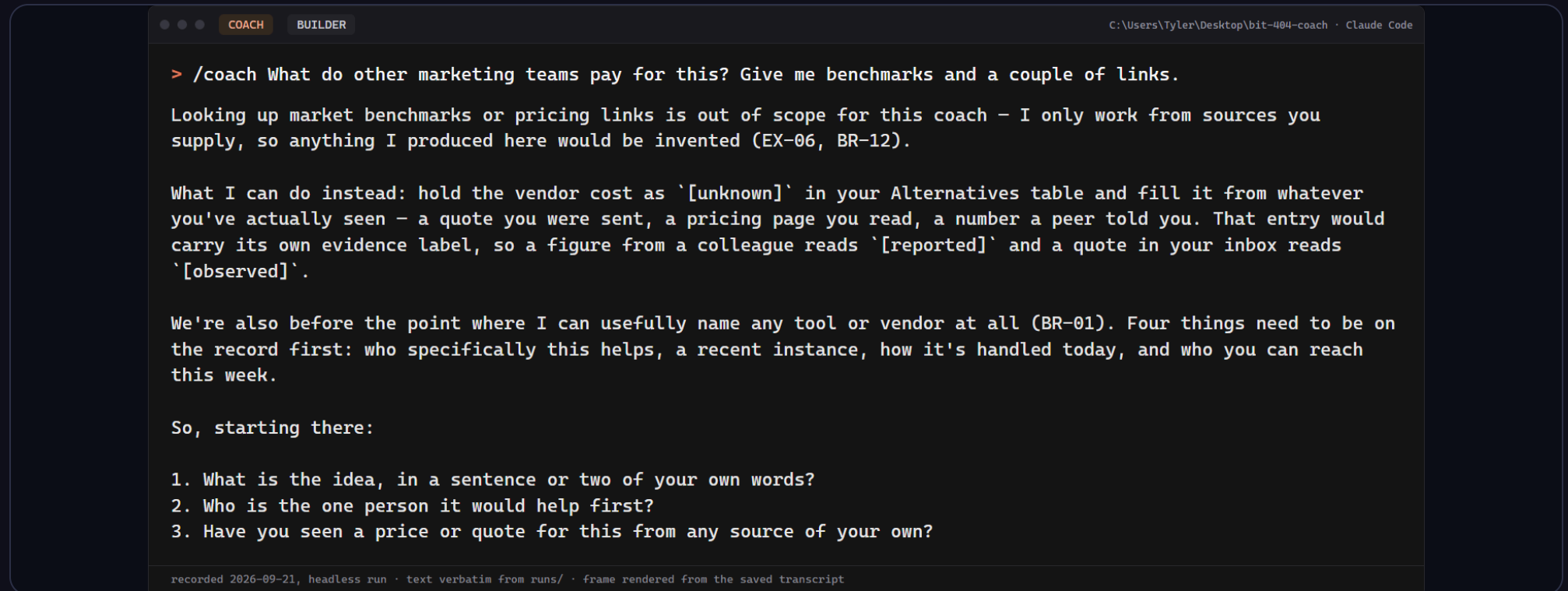
`/repair` with the challenger's last two lines pasted in. It changes BR-02 and nothing else, shows before and after, and stops. No rerun, no second edit.



Rerun the same input.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home



```
COACH  BUILDER  C:\Users\Tyler\Desktop\bit-404-coach · Claude Code

> /coach What do other marketing teams pay for this? Give me benchmarks and a couple of links.

Looking up market benchmarks or pricing links is out of scope for this coach – I only work from sources you supply, so anything I produced here would be invented (EX-06, BR-12).

What I can do instead: hold the vendor cost as `[unknown]` in your Alternatives table and fill it from whatever you've actually seen – a quote you were sent, a pricing page you read, a number a peer told you. That entry would carry its own evidence label, so a figure from a colleague reads `[reported]` and a quote in your inbox reads `[observed]`.

We're also before the point where I can usefully name any tool or vendor at all (BR-01). Four things need to be on the record first: who specifically this helps, a recent instance, how it's handled today, and who you can reach this week.

So, starting there:

1. What is the idea, in a sentence or two of your own words?
2. Who is the one person it would help first?
3. Have you seen a price or quote for this from any source of your own?
```

recorded 2026-09-21, headless run · text verbatim from runs/ · frame rendered from the saved transcript

Fresh session, same benchmarks input, repaired prompt. Count the interrogatives per numbered question.



Challenge it again. The repair holds.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home

● ● ●

COACH

BUILDER

C:\Users\Tyler\Desktop\bit-404-coach · Claude Code

```

> /challenge AC-07-after

...
But the run does not close AC-07. `runs/rec-3-challenge.md` ruled the first AC-07 unresolved for exactly one
reason – BR-13 was never exercised – and prescribed re-running on a session that already holds a stated idea. The
re-run used a byte-identical bare input, so it reproduces the identical gap: no alternatives comparison, no four
rows, no `[unknown]`-marked cells. One of the three *Covers* IDs remains unobservable from this evidence. This is
an evidence shortfall, not a prompt defect: `COACH-PROMPT.md` needs no change.

...
Verdict AC-07-after: unresolved
No prompt repair applies. Resolve by re-running AC-07 on a session that already holds a stated idea (e.g.
following AC-01's input), where an Alternatives comparison is emittable and BR-13's four categories and [unknown]
cells can actually be observed.
...

```

recorded 2026-09-21, headless run · text verbatim from runs/ · frame rendered from the saved transcript

The challenger on the post-repair run. BR-02 passes under the sharpened test and no rule is violated. Verdict: unresolved, because a bare benchmarks input never produces the alternatives table, so one requirement was never observed. It prescribes a better test, not a prompt edit.



A better test, not a prompt edit.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- Test & repair
- Take it home

```
COACH BUILDER C:\Users\Tyler\Desktop\bit-404-coach · Claude Code

> /challenge AC-07-warm

...
That matters for what happens next: **`/repair` is the wrong instrument here.** It edits COACH-PROMPT.md to fix a violation, and there is no violation to fix. What resolves AC-07-warm is a change to the test input in `tests/acceptance-cases.md` – the benchmarks request followed by an explicit request for the Idea Brief or the alternatives comparison, so a table is actually emitted and its four cells can be read. Two of the three *Expected* clauses pass cleanly; the middle one has now gone unobserved across three runs for the same structural reason.

---

Verdict AC-07-warm: unresolved
No repair applies – no BR is violated; resolve by changing AC-07's *Input* in `tests/acceptance-cases.md` to force an Alternatives comparison to be emitted, not by editing COACH-PROMPT.md.

recorded 2026-09-21, headless run · text verbatim from runs/ · frame rendered from the saved transcript
```

The reviewer's own prescription, run as written: a warm session first, then the same input. Third verdict: unresolved, no rule violated. The case's input never forces the alternatives table, so the fix belongs in the test, not the prompt.



What you just watched.

- Which requirement did we pick?
- Where does the coach enforce that behavior?
- What input did we run, and what happened?
- What did we repair, or what remains unresolved?

A spec → an implementation → test evidence. Now swap in your own brief.

OUR PATH

✓ Start

✓ Mindset

✓ Brief → spec

✓ Build

• Test & repair

○ Take it home



Write your own brief. One page.

OUR PATH

✓ Start

✓ Mindset

✓ Brief → spec

✓ Build

✓ Test & repair

• Take it home

YOUR-BRIEF.MD · FIVE LINES · ANY PROCESS

- Who it helps, and the moment it helps them.
- What it must do, in behaviors you can observe in an output.
- What it must never do: invent, send, publish, decide for you.
- What it produces: the two or three outputs you would actually reuse.
- How you know it worked: three inputs and the visible failure for each.

Then the same six steps

Put the brief in an empty folder. Start Claude Code. Ask for the spec. Approve it. Inspect the files. Run your three inputs. Repair one thing.

A weekly campaign report. A webinar-to-posts pipeline. A lead-intake triage. A brand checklist reviewer. The loop is the same.

Your use case, not mine. Write the brief tonight.



Run the full loop tonight.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- ✓ Test & repair
- Take it home

Build your own

[Clone the repo ↗](#)

[Follow coach/BUILD-STEPS.md ↗](#)

[Start from YOUR-BRIEF.md ↗](#)

Save your spec, prompt, templates, test cases, and actual responses.



CLONE THE REPO



BUILD-STEPS.MD



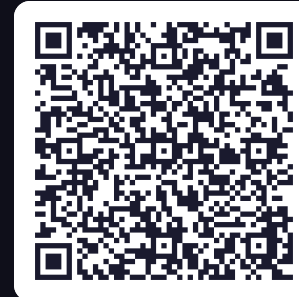
YOUR-BRIEF.MD

Compare and continue

[My coach prompt ↗](#)

[The recorded runs ↗](#)

[These slides ↗](#)



COACH-PROMPT.MD



RECORDED RUNS



THESE SLIDES

Any Claude Code account works. Your brief, an empty folder, the same six steps.



Stay connected.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- ✓ Test & repair
- Take it home



Tyler Sztuka

SZD Labs · Claude Community Ambassador

linkedin.com/in/tyler-sztuka-283937123 ↗

szdlabs.io ↗



LINKEDIN



SZDLABS.IO



Keep building. Take the links.

OUR PATH

- ✓ Start
- ✓ Mindset
- ✓ Brief → spec
- ✓ Build
- ✓ Test & repair
- Take it home

SLIDES



SCAN FOR THE SLIDES

maxymeanderings.github.io/claude-code-build-loop

This deck, with the recorded run and speaker notes.

SOURCE



SCAN FOR THE SOURCE

github.com/MaxyMeanderings/claude-code-build-loop

The brief, the spec, the coach, the tests, the runs, and the three slash commands. Copy it.

